



MODULE 01/09

Git & GitHub

Contrôle de version et collaboration

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre le concept de versionnage et pourquoi c'est indispensable
 - ✓ Maîtriser les commandes Git du quotidien (init, add, commit, push, pull)
 - ✓ Travailler avec des branches pour isoler ses fonctionnalités
 - ✓ Collaborer via GitHub : pull requests, code review
 - ✓ Lire et écrire un historique Git propre sur Social Seller
-

Planning de la semaine

JOUR 1	Concepts fondamentaux	2-3h
	• Qu'est-ce que le versionnage ? Pourquoi Git et pas autre chose ? • Le modèle Git : working directory → staging area → repository • Installation et configuration initiale (git config --global) • Commandes de base : git init, git status, git log	
JOUR 2	Commits et historique	2-3h
	• git add, git commit -m — écrire de bons messages de commit • git log --oneline --graph — lire l'historique • git diff — voir les modifications avant de committer • git restore / git reset — annuler des erreurs	
JOUR 3	Branches et merge	2-3h
	• Pourquoi des branches ? Stratégie feature branch • git branch, git checkout -b, git switch • git merge vs git rebase — quand utiliser quoi • Résoudre un conflit de merge manuellement	
JOUR 4	GitHub et collaboration	2-3h
	• Créer un repo GitHub, git remote add origin, git push • git clone, git pull, git fetch • Créer une Pull Request, demander une review • GitHub Actions — introduction aux pipelines CI	
JOUR 5	Mini-projet de la semaine	3-4h
	• Créer un repo GitHub 'mon-portfolio' depuis zéro • Faire 5 commits sur main avec messages conventionnels	

- Créer une branche feature/contact, ajouter une page contact
- Merger la branche via une Pull Request sur GitHub
- Taguer la version v1.0.0

Concepts clés détaillés

Le modèle Git en 3 zones

Git organise ton travail en 3 zones distinctes. Comprendre ce modèle est la clé pour ne jamais être perdu.

```
Working Directory → Staging Area → Repository
(tes fichiers) (git add) (git commit)

# Vérifier l'état de chaque zone
git status

# Ajouter un fichier au staging
git add mon-fichier.ts

# Ajouter TOUS les fichiers modifiés
git add -A

# Créer un commit
git commit -m "feat: ajouter page de connexion"
```

■ *Convention de message de commit : feat: (nouvelle fonctionnalité), fix: (correction), chore: (maintenance), docs: (documentation). Exemple : 'feat: Sprint 3 — OAuth WhatsApp'*

Branches — la commande essentielle

```
# Créer et basculer sur une nouvelle branche
git checkout -b feature/ma-fonctionnalite

# (équivalent moderne)
git switch -c feature/ma-fonctionnalite

# Lister toutes les branches
git branch -a

# Revenir sur main
git switch main

# Merger la branche dans main
git merge feature/ma-fonctionnalite

# Supprimer la branche une fois mergée
```

```
git branch -d feature/ma-fonctionnalite
```

Résoudre un conflit de merge

Un conflit arrive quand deux branches modifient la même ligne. Git marque le fichier :

```
<<<<<< HEAD (ta branche courante)
const message = 'Bonjour';
=====
const message = 'Hello';
>>>>>> feature/traduction

# Étapes pour résoudre :
# 1. Ouvrir le fichier, choisir quelle version garder
# 2. Supprimer les marqueurs <<<< ==== >>>>
# 3. git add le-fichier.ts
# 4. git commit -m "fix: résoudre conflit traduction"
```

Workflow GitHub quotidien

```
# 1. Partir à jour de main
git switch main && git pull

# 2. Créer ta branche
git switch -c feat/nouveau-ecran

# 3. Coder... puis committer régulièrement
git add -A && git commit -m 'feat: ajouter écran stats'

# 4. Pousser vers GitHub
git push origin feat/nouveau-ecran

# 5. Ouvrir une Pull Request sur GitHub
# → Bouton 'Compare & pull request'

# 6. Après merge, nettoyer
git switch main && git pull
git branch -d feat/nouveau-ecran
```

Exercices pratiques

Exercice 1 — Premier repo (30 min)

- Créer un dossier 'exercice-git', y mettre 3 fichiers .txt

- Initialiser un repo Git et faire un premier commit
- Modifier un fichier, vérifier git diff, committer
- Regarder git log --oneline

Exercice 2 — Branches (45 min)

- Dans le même repo, créer une branche 'feature/page-2'
- Ajouter un fichier page2.txt, committer
- Revenir sur main, constater que page2.txt n'y est pas
- Merger la branche, constater que page2.txt apparaît

Exercice 3 — Conflit volontaire (45 min)

- Créer deux branches qui modifient la même ligne du même fichier
 - Merger la première branche dans main
 - Tenter de merger la deuxième — observer le conflit
 - Résoudre le conflit manuellement et terminer le merge
-

Ressources pour aller plus loin

- Documentation officielle : git-scm.com/doc
- Visualiseur interactif : learngitbranching.js.org
- Conventional Commits : conventionalcommits.org
- GitHub Docs : docs.github.com/fr
- Oh Shit, Git! : ohshitgit.com/fr — pour réparer les erreurs courantes